

INFO6002 DATABASE MANAGEMENT 2
ASSIGNMENT 1

Group Members:

Quynh Anh Nguyen c3473095

Leiming Guo c3471779

Shama Praveen c3470623

1.1 EER Model



Entity Name	Description	Occurrence
Customer	Individual renting vehicles, having contact and license info.	Each customer uniquely identified
LoyaltyTier	Customer loyalty level providing benefits like discounts.	Zero or one tier per customer
Employee	Staffs working in a branch, handling bookings, service, and return service	Each employee belongs to one branch

Branch	Company location managing vehicles and employees.	Unique per location
Vehicle	Car available for rent, with details like make, model, and status.	Each vehicle belongs to one branch
VehicleCategory	Classification of vehicles affecting pricing.	Each vehicle assigned exactly one
Service	Rental service type offered (base entity for regular and package services).	Abstract/general entity
RegularService	Regular rental options like Self-Drive, Chauffeur-Driven.	Specialized type of Service
PackageService	Bundled offers combining vehicles and inclusions/exclusions.	Specialized type of Service
PricedService	Specific pricing of a service at a branch for a period.	Tied to branch and service
Booking	Reservation made by a customer for one or more vehicles and services.	Unique per customer request
Driver	Individuals authorized to drive vehicles on a booking.	Zero or more per booking
BookingVehicle	Vehicle reserved under a booking.	Each booking may reserve multiple
BookingService	Service included in a booking.	Each booking can include multiple
ExtraService	Add-ons like child seats, navigators, insurance upgrades.	Optional services added to bookings
BookingExtraService	Extra services added to a booking.	Each booking can include multiple
ReturnDetails	Information recorded when vehicle is returned (time, condition).	Zero or one per booking
Invoice	Billing record including all charges, discounts, and payments.	One per completed rental
Discount	Reduction applied to invoice, usually for loyalty or manager approval.	Zero or one per invoice

1.2.2 Relationship

Entity Name	Multiplicity	Relationship	Multiplicity	Entity Name
Customer	1..1	assigned to	0..1	LoyaltyTier
Customer	1..1	makes	0..*	Booking
Branch	1..1	employs	0..*	Employee
Employee	1..1	works at	1..1	Branch
Branch	1..1	owns	0..*	Vehicle
Vehicle	1..1	belongs to	1..1	Branch

Vehicle	1..1	classified as	1..1	VehicleCategory
Service	0..1	generalizes	1..1	RegularService
Service	0..1	generalizes	1..1	PackageService
Service	1..1	used in	1..*	PricedService
Branch	1..1	offers	0..*	PricedService
PricedService	1..1	approved by	1..1	Employee (Manager)
Booking	1..1	includes	1..*	BookingVehicle
BookingVehicle	1..1	reserves	1..1	Vehicle
Booking	1..1	requests	1..*	BookingService
BookingService	1..1	refers to	1..1	PricedService
Booking	0..*	includes	0..*	Driver
Booking	0..*	adds	0..*	BookingExtraService
BookingExtraService	1..1	refers to	1..1	ExtraService
Booking	1..1	handled by	1..1	Employee (any role)
Booking	0..1	returns	0..1	ReturnDetails
ReturnDetails	1..1	finalised by	1..1	Employee (any role)
ReturnDetails	1..1	generates	1..1	Invoice
Invoice	1..1	has	0..1	Discount
Discount	0..1	approved by	1..1	Employee (Manager)

1.2.3 Attributes

Customer:

Attribute	Description	Data Type	Nulls	Multivalued	Derived	Default
CustomerID	Unique ID for the customer	INT (PK)	No	No	No	—
FullName	Full legal name	VARCHAR(100)	No	No	No	—
Email	Email address	VARCHAR(100)	No	No	No	—
Phone	Contact number	VARCHAR(20)	Yes	No	No	—
DriverLicenseNo	Driving license number	VARCHAR(30)	No	No	No	—
Country	Country of residence	VARCHAR(50)	Yes	No	No	—
LoyaltyTierID	FK to Loyalty Tier	INT (FK)	Yes	No	No	NULL

Loyalty Tier:

Attribute	Description	Data Type	Nulls	Multivalued	Derived	Default
LoyaltyTierID	Unique ID	INT (PK)	No	No	No	–
TierName	Tier name (Silver, Gold, etc.)	VARCHAR(20)	No	No	No	–
DiscountRate	Discount percentage	DECIMAL(5, 2)	No	No	No	–

Employee

Entity	Attributes	Description	Data Type & Length	Nulls	Multivalued	Derived	Default
Employee	EmployeeID	Unique identifier	Integer	No	No	No	Auto
Employee	BranchID (FK)	Branch where employee works	Integer	No	No	No	-
Employee	Name	Full name	Varchar(100)	No	No	No	-
Employee	Email	Work email	Varchar(100)	Yes	No	No	
Employee	Phone	Contact number	Varchar(15)	Yes	No	No	
Employee	Position	Manager or Associate	Enum	No	No	No	

Branch:

Entity	Attributes	Description	Data Type & Length	Nulls	Multivalued	Derived	Default
Branch	BranchID	Unique branch ID	Integer	No	No	No	Auto
Branch	Name	Branch name	Varchar(100)	No	No	No	
Branch	Location	Physical address	Varchar(200)	No	No	No	
Branch	OperationalStatus	Active, Inactive, or Closed	Enum	No	No	No	

Vehicles Category:

Entity	Attributes	Description	Data Type & Length	Nulls	Multivalued	Derived	Default
VehicleCategory	CategoryID	Unique identifier for the vehicle category	Integer	No	No	No	Auto
VehicleCategory	Name	Category name (e.g., Economy, Standard, Luxury)	Enum	No	No	No	
VehicleCategory	Description	Text description of the category features	String/Text	Yes	No	No	
VehicleCategory	DailyRateMultiplier	Used to calculate total price from base rental rate	Float	No	No	No	1.0

Vehicles

Entity	Attributes	Description	Data Type & Length	Nulls	Multivalued	Derived	Default
Vehicle	VehicleID	Unique vehicle identifier	Integer	No	No	No	Auto
Vehicle	Make	Manufacturer (e.g., Toyota, Ford)	Varchar(50)	No	No	No	
Vehicle	Model	Model name	Varchar(50)	No	No	No	
Vehicle	Year	Manufacturing year	Integer	No	No	No	
Vehicle	Color	Vehicle color	Varchar(30)	Yes	No	No	
Vehicle	LicensePlateNumber	Vehicle registration/license number	Varchar(20)	No	No	No	
Vehicle	Status	Active, Inactive, Decommissioned	Enum	No	No	No	
Vehicle	FuelType	Diesel, Petrol, Electric	Enum	No	No	No	
Vehicle	Transmission Type	Manual, Automatic	Enum	No	No	No	
Vehicle	BaseRentalRate	Base daily rental rate	Decimal (10,2)	No	No	No	
Vehicle	BranchID (FK)	Branch that owns the vehicle	Integer	No	No	No	
Vehicle	CategoryID (FK)	Category of the vehicle	Integer	No	No	No	

Booking

Entity	Attributes	Description	Data Type & Length	Nulls	Multivalued	Derived	Default
Booking	BookingID	Unique identifier for each booking	Integer	No	No	No	Auto
Booking	BookingDate	Date the booking	Date	No	No	No	Current Date

		was made					
Bookin g	CustomerID (FK)	The custome r who made the booking	Integer	No	No	No	
Bookin g	BookingStat us	Status of the booking (e.g., Confirm ed, Cancell ed, Service d)	Enum	No	No	No	
Bookin g	PaymentMet hod	Method of paymen t (e.g., Credit Card, Cash, Bank Transfer)	Varchar(30)	Yes	No	No	
Bookin g	Deposit	Deposit amount paid at the time of booking	Decimal(1 0,2)	Yes	No	No	0.00
Bookin g	HandledBy (FK)	Employ ee who handled the booking	Integer	Yes	No	No	

ReturnDetails

Entity	Attributes	Descriptio n	Data Type & Length	Nulls	Multival ued	Derived	Default
ReturnDeta ils	ReturnID	Unique identifier for the return process	Integer	No	No	No	Auto

ReturnDetails	BookingID (FK)	Associated booking ID	Integer	No	No	No	
ReturnDetails	ActualPickupTime	Actual time the vehicle was picked up	Datetime	No	No	No	
ReturnDetails	DropOffTime	Actual time the vehicle was returned	Datetime	No	No	No	
ReturnDetails	VehicleCondition	Condition of the vehicle upon return (e.g., damages, clean status)	Text	Yes	No	No	
ReturnDetails	LatePenalty	Penalty charged for returning the vehicle late	Decimal(10,2)	Yes	No	No	0.00
ReturnDetails	DamagePenalty	Penalty for vehicle damage	Decimal(10,2)	Yes	No	No	0.00
ReturnDetails	FinalAmount	Final amount to be paid after adjustments	Decimal(10,2)	No	No	Yes	
ReturnDetails	ProcessedBy (FK)	Employee who processed the return	Integer	No	No	No	
ReturnDetails	DiscountApprover (FK)	Manager who approved the discount if not a	Integer	Yes	No	No	

		loyal customer					
--	--	-------------------	--	--	--	--	--

Service:

Entity	Attributes	Description	Data Type & Length	Nulls	Multivalued	Derived	Default
Service	ServiceID	Unique identifier for the service	Integer	No	No	No	Auto
Service	Name	Name of the service	Varchar(100)	No	No	No	
Service	Description	Brief description of the service	Text	Yes	No	No	
Service	Status	Availability status (e.g., Available, Unavailable)	Enum	No	No	No	
Service	Type	Indicates whether it's Regular or Package	Enum	No	No	No	

1.2.4 Type Hierarchy

Parent Entity	Child Entities (Subtypes)	Reason / Explanation
Employee	Manager, Associate	Different roles with different authority levels
Service	RegularService, PackageService	Two types of services with different attributes
VehicleCategory	Economy, Standard, Luxury, Commercial	Categories classify vehicles with different features
Vehicle	(Optional: can be specialized by category)	Could be further detailed if needed (e.g., SUV, Sedan)

1. Employee

Parent: Employee

Child: Associate, Manager

Explanation:

- The common information between the two types of employees is: EmployeeID, Name, Email, Phone, BranchID, and Position.
- Manager is set apart by greater authority, including authorisation of discounts and services at a price.
- Associate shall be allowed to do booking and returns only, and discounts should be offered only to loyal customers.
- Subtyping enhances stored procedure and role-based logic validations.

2. Service

Parent: Service

Child: RegularService, PackageService

Explanation:

- The common attributes of all the services include ServiceID, Name, Description, and Status.
- RegularService has other attributes such as BasePrice, Currency and Restrictions.
- PackageService contains BasePrice, Currency, ValidPeriodStart, ValidPeriodEnd, Inclusions, Exclusions and GracePeriod.
- Such generalization Favors improved extensibility and normalization of service types.

3. VehicleCategory

Parent: VehicleCategory

Child Categories (Static Values): Economy, Standard, Luxury, Commercial

Explanation:

- Though they are not real sub-entities in schema, they are predefined types (a domain set).
- Each type has an impact on the rental regulations (e.g. Luxury Chauffeur-Driven only, Commercial Fleet-Leasing)

ASSUMPTIONS

Branches

- A branch may have more than one employee, vehicle and service.
- The status of a branch can either be Active, Inactive or Closed.

Employees

All the employees are based in a single branch.

- Discounts and service prices can be approved only by Managers.
- Employees are either Managers or Associates (ISA hierarchy).

Vehicles

- Every vehicle is of one branch and one category of vehicle.
- Vehicle status: Active/Inactive/Decommissioned.
- Enums are used in vehicles: FuelType and TransmissionType.

Vehicle Categories

- Specified as look up values: Economy, Standard, Luxury, Commercial.
- Pricing multipliers, not modeled as subclasses.

Customers

- A customer can or can not have Loyalty Tier.
- Clients will be able to reserve several cars and services.
- Discount percentages are defined by Loyalty Tiers.

Services

- Services will be Regular or Package (ISA).
- Services have to be priced per individual branches (through PricedService).
- Service up / down: Up or Down.

PricedService

- Pricing by branch and service valid date range.
- Requires Manager approval.

Bookings

- Every reservation is made by a client and processed by a staff member.
- A reservation may consist of many vehicles, services, additional services, and drivers.
- One return record and one invoice against each booking.

Drivers

- Optional on booking (e.g. self-drive), more than one driver.

Return Details

- Comments on state, time of returning and fines.
- Cleared and discount-allowed by workers.

Invoices

- Produced at the booking level, it comprises charges and discount applied.

Enums / Lookup Tables Used.

- Vehicle.Status, Branch.Status, Service.Status, Booking.Status
- TransmissionType, FuelType, TierName (e.g., Silver, Gold, Platinum)

❖ Task 2: Map the EER Diagram to Relational Model

Customer(customerID, name, email, phone, TierID, driving liscenseNo, Country)

Primary key customerID

Foreign key loyaltyTierID references LoyaltyTier(loyaltyTierID)

For LoyaltyTier

LoyaltyTier(TierID, DiscountRate)

Primary key TierID

For Branch:

Branch(branchID, location, status)

Primary key branchID

For Employee:

Employee(employeeID, name, email, position, phone, branchID)

Primary key employeeID

Foreign key branchID references Branch(branchID)

For Vehicle:

Vehicle(vehicleID, make, model, categoryID, bookingID, Year, Color, LicensePlateNumber, Status)

Primary key vehicleID

Foreign key categoryID references VehicleCategory(categoryID)

Foreign key branchID references Booking(bookingID)

For VehicleCategory:

VehicleCategory(categoryID, name, DailyRateMultiplier)

Primary key categoryID

For Service:

Service(serviceID, name, description,status,sype)

Primary key servicedID

For RegularService:

RegularService(serviceID, basedPriced, Currency, Retrictions)

Primary key serviceID

Foreign key serviceID references Service(serviceID)

For PackageService:

PackageService(serviceID, BasePrice, Currency, ValidStart, ValidEnd, Inclusions,
Exclusions, GracePeriod)

Primary key serviceID

Foreign key serviceID references Service(serviceID)

For PricedService:

PricedService(PricedServiceID, BranchID, Price, StartDate, EndDate, Currency, ApproverID)

Primary key pricedServiceID

Foreign key serviceID references Service(serviceID)

Foreign key branchID references Branch(branchID)

Foreign key approvedBy references Employee(employeeID)

For Booking:

Booking(BookingDate, CustomerID, BookingStatus, PaymentMethod, Deposit, HandledBy)

Primary key bookingID

Foreign key customerID references Customer(customerID)

Foreign key handledBy references Employee(employeeID)

For BookingVehicle:

BookingVehicle (bookingID, vehicleID)

Foreign key bookingID references Booking(bookingID)

Foreign key vehicleID references Vehicle(vehicleID)

For BookingService:

BookingService(bookingID, pricedServiceID, ServiceStart, ServiceEnd)

Foreign key bookingID references Booking(bookingID)

Foreign key pricedServiceID references PricedService(pricedServiceID)

For ExtraService

ExtraService(ExtraServiceID, Name, AdvertisedRate)

Primary key ExtraServiceID references ExtraService (ExtraServiceID)

For BookingExtraService:

BookingExtraService (BookingID, ExtraServiceID)

Foreign key bookingID references Booking(bookingID)

Foreign key extraService references ExtraService(ExtraServiceID)

For ReturnDetails:

ReturnDetails(BookingID, ActualPickupTime, DropOffTime, VehicleCondition, LatePenalty, DamagePenalty, FinalAmount, ProcessedBy, DiscountApprover)

Primary key returnID

Foreign key bookingID references Booking(bookingID)

Foreign key finalised By references Employee(employeeID)

For Invoice:

Invoice(BookingID, BaseCharge, ExtraServiceCharge, LatePenalty, DamagePenalty, Discount, FinalAmount, PaymentMethod)

Primary key invoiceID

Foreign key returnID references ReturnDetails(returnID)

Foreign key discountID references Discount(discountID)

❖ Task 2: List the Functional Dependencies (FDs)

For customer:

customerID $\rightarrow$ name, email, phone, TierID, drivingLicenseNo, Country

TierID $\rightarrow$ DiscountRate (via LoyaltyTier)

For LoyaltyTier:

TierID $\rightarrow$ DiscountRate

For Branch:

branchID $\rightarrow$ location, status

For Employee:

employeeID $\rightarrow$ name, email, position, phone, branchID

For Vehicle:

vehicleID $\rightarrow$ make, model, categoryID, bookingID, Year, Color, LicensePlateNumber, Status

categoryID $\rightarrow$ name, DailyRateMultiplier (via VehicleCategory)

For VehicleCategory

categoryID $\rightarrow$ name, DailyRateMultiplier

For Service

serviceID $\rightarrow$ name, description, status, type

For regularService:

serviceID $\rightarrow$ basePrice, currency, restrictions

For packageService:

serviceID $\rightarrow$ basePrice, currency, validStart, validEnd, inclusions, exclusions, gracePeriod

For pricedService:

pricedServiceID $\rightarrow$ branchID, price, startDate, endDate, currency, approverID

For Booking:

bookingID $\rightarrow$ bookingDate, customerID, bookingStatus, paymentMethod, deposit, handledBy

For BookingVehicle:

(bookingID, vehicleID) $\rightarrow \emptyset$

For BookingService:

(bookingID, pricedServiceID) $\rightarrow$ serviceStart, serviceEnd

For ExtraService:

extraServiceID $\rightarrow$ name, advertisedRate

For BookingExtraService:

(bookingID, extraServiceID) $\rightarrow \emptyset$

For Return Details:

returnID $\rightarrow$ bookingID, actualPickupTime, dropOffTime, vehicleCondition, latePenalty, damagePenalty, finalAmount, processedBy, discountApprover

For Invoice:

invoiceID $\rightarrow$ bookingID, baseCharge, extraServiceCharge, latePenalty, damagePenalty, discount, finalAmount, paymentMethod

❖ Task 2: Check for BCNF & Normalize if Needed

1. Customer(customerID, name, email, phone, TierID, drivingLicenseNo, Country)

FDs:

customerID $\rightarrow$ name, email, phone, TierID, drivingLicenseNo, Country

TierID $\rightarrow$ DiscountRate (via LoyaltyTier)

Analysis:

All non-key attributes are fully functionally dependent on the primary key customerID.

BCNF satisfied.

2. LoyaltyTier(TierID, DiscountRate)

FD: TierID $\rightarrow$ DiscountRate

BCNF satisfied.

3. Branch(branchID, location, status)

FD: branchID $\rightarrow$ location, status

BCNF satisfied.

4. Employee(employeeID, name, email, position, phone, branchID)

FD: employeeID $\rightarrow$ name, email, position, phone, branchID

BCNF satisfied.

5. Vehicle(vehicleID, make, model, categoryID, bookingID, year, color, licensePlateNumber, status)

FDs:

vehicleID $\rightarrow$ make, model, categoryID, bookingID, year, color, licensePlateNumber, status

categoryID $\rightarrow$ name, DailyRateMultiplier (via VehicleCategory)

Issue:

Including bookingID inside the Vehicle table implies a one-to-one relationship, but each vehicle can be booked many times over time. This introduces a transitive and multivalued dependency. So it violates BCNF.

Resolution:

Remove bookingID from Vehicle. Store this relationship in the existing BookingVehicle table.

Decomposition:

Vehicle(vehicleID, make, model, categoryID, year, color, licensePlateNumber, status)

BookingVehicle(bookingID, vehicleID)

Now satisfies BCNF.

6. VehicleCategory(categoryID, name, DailyRateMultiplier)

FD: categoryID $\rightarrow$ name, DailyRateMultiplier

BCNF satisfied.

7. Service(serviceID, name, description, status, type)

FD: serviceID $\rightarrow$ name, description, status, type

BCNF satisfied.

8. RegularService(serviceID, basePrice, currency, restrictions)

FD: serviceID $\rightarrow$ basePrice, currency, restrictions

BCNF satisfied.

9. PackageService(serviceID, basePrice, currency, validStart, validEnd, inclusions, exclusions, gracePeriod)

FD: serviceID $\rightarrow$ all other attributes

BCNF satisfied.

10. PricedService(pricedServiceID, branchID, price, startDate, endDate, currency, approverID)

FD: pricedServiceID $\rightarrow$ all other attributes

BCNF satisfied.

11. Booking(bookingID, bookingDate, customerID, bookingStatus, paymentMethod, deposit, handledBy)

FD: bookingID $\rightarrow$ all other attributes

BCNF satisfied.

12. BookingVehicle(bookingID, vehicleID)

FD: (bookingID, vehicleID) $\rightarrow$ Associative entity with no non-key attributes.

BCNF satisfied.

13. BookingService(bookingID, pricedServiceID, serviceStart, serviceEnd)

FD: (bookingID, pricedServiceID) $\rightarrow$ serviceStart, serviceEnd

BCNF satisfied.

14. ExtraService(extraServiceID, name, advertisedRate)

FD: extraServiceID $\rightarrow$ name, advertisedRate

BCNF satisfied.

15. BookingExtraService(bookingID, extraServiceID)

FD: (bookingID, extraServiceID) $\rightarrow$ Associative entity with no non-key attributes.

BCNF satisfied.

16. ReturnDetails(returnID, bookingID, actualPickupTime, dropOffTime, vehicleCondition, latePenalty, damagePenalty, finalAmount, processedBy, discountApprover)

FD: returnID $\rightarrow$ all other attributes

BCNF satisfied.

17. Invoice(invoiceID, bookingID, baseCharge, extraServiceCharge, latePenalty, damagePenalty, discount, finalAmount, paymentMethod)

FD: invoiceID $\rightarrow$ all other attributes

BCNF satisfied.

Summary of Normalisation

Table	Normalised?	Notes
Customer	YES	Fully dependent on PK
LoyaltyTier	YES	Simple dependency
Branch	YES	Fully functionally dependent
Employee	YES	No transitive dependencies
Vehicle	NO	bookingID removed; normalised with BookingVehicle
VehicleCategory	YES	Fully dependent on PK
Service	YES	Fully dependent on PK
RegularService	YES	Subclass; no violation
PackageService	YES	Subclass; no violation
PricedService	YES	Fully dependent on PK
Booking	YES	Fully dependent on PK
BookingVehicle	YES	Associative table
BookingService	YES	Composite key; no violation
ExtraService	YES	Simple dependency
BookingExtraService	YES	Associative table
ReturnDetails	YES	Fully dependent on PK
Invoice	YES	Fully dependent on PK

❖ Task 2: Write Final Schema in DBDL Format

Customer(customerID, name, email, phone, TierID, drivingLicenseNo, country)

Primary Key customerID

Foreign Key TierID references LoyaltyTier(TierID)

ON UPDATE CASCADE, ON DELETE SET NULL

LoyaltyTier(TierID, DiscountRate)

Primary Key TierID

Branch(branchID, location, status)

Primary Key branchID

Employee(employeeID, name, email, position, phone, branchID)

Primary Key employeeID

Foreign Key branchID references Branch(branchID)

ON UPDATE CASCADE, ON DELETE SET NULL

VehicleCategory(categoryID, name, DailyRateMultiplier)

Primary Key categoryID

Vehicle(vehicleID, make, model, categoryID, year, color, licensePlateNumber, status)

Primary Key vehicleID

Foreign Key categoryID references VehicleCategory(categoryID)

ON UPDATE CASCADE, ON DELETE SET NULL

Service(serviceID, name, description, status, type)

Primary Key serviceID

RegularService(serviceID, basePrice, currency, restrictions)

Primary Key serviceID

Foreign Key serviceID references Service(serviceID)

ON UPDATE CASCADE, ON DELETE CASCADE

PackageService(serviceID, basePrice, currency, validStart, validEnd, inclusions, exclusions, gracePeriod)

Primary Key serviceID

Foreign Key serviceID references Service(serviceID)

ON UPDATE CASCADE, ON DELETE CASCADE

PricedService(pricedServiceID, branchID, price, startDate, endDate, currency, approverID)

Primary Key pricedServiceID

Foreign Key branchID references Branch(branchID)

ON UPDATE CASCADE, ON DELETE CASCADE

Foreign Key approverID references Employee(employeeID)

ON UPDATE CASCADE, ON DELETE SET NULL

Booking(bookingID, bookingDate, customerID, bookingStatus, paymentMethod, deposit, handledBy)

Primary Key bookingID

Foreign Key customerID references Customer(customerID)

ON UPDATE CASCADE, ON DELETE CASCADE

Foreign Key handledBy references Employee(employeeID)

ON UPDATE CASCADE, ON DELETE SET NULL

BookingVehicle(bookingID, vehicleID)

Primary Key bookingID, vehicleID

Foreign Key bookingID references Booking(bookingID)

ON UPDATE CASCADE, ON DELETE CASCADE

Foreign Key vehicleID references Vehicle(vehicleID)

ON UPDATE CASCADE, ON DELETE CASCADE

BookingService(bookingID, pricedServiceID, serviceStart, serviceEnd)

Primary Key bookingID, pricedServiceID

Foreign Key bookingID references Booking(bookingID)

ON UPDATE CASCADE, ON DELETE CASCADE

Foreign Key pricedServiceID references PricedService(pricedServiceID)

ON UPDATE CASCADE, ON DELETE CASCADE

ExtraService(extraServiceID, name, advertisedRate)

Primary Key extraServiceID

BookingExtraService(bookingID, extraServiceID)

Primary Key bookingID, extraServiceID

Foreign Key bookingID references Booking(bookingID)

ON UPDATE CASCADE, ON DELETE CASCADE

Foreign Key extraServiceID references ExtraService(extraServiceID)

ON UPDATE CASCADE, ON DELETE CASCADE

ReturnDetails(bookingID, actualPickupTime, dropOffTime, vehicleCondition, latePenalty, damagePenalty, finalAmount, processedBy, discountApprover)

Primary Key bookingID

Foreign Key bookingID references Booking(bookingID)

ON UPDATE CASCADE, ON DELETE CASCADE

Foreign Key processedBy references Employee(employeeID)

ON UPDATE CASCADE, ON DELETE SET NULL

Foreign Key discountApprover references Employee(employeeID)

ON UPDATE CASCADE, ON DELETE SET NULL

Invoice(invoiceID, bookingID, baseCharge, extraServiceCharge, latePenalty, damagePenalty, discount, finalAmount, paymentMethod)

Primary Key invoiceID

Foreign Key bookingID references Booking(bookingID)

ON UPDATE CASCADE, ON DELETE CASCADE

❖ Task 3: Database Implementation

GlobalCarHire Database Schema (DDL)

```
1. Create Table
2. -- Drop existing tables if they exist (in correct order to handle foreign keys)
3. DROP TABLE IF EXISTS Invoice, ReturnDetails, BookingExtraService, ExtraService,
   Driver,
4.   BookingVehicle, BookingService, Booking, PricedService, PackageService,
   RegularService,
5.   Service, Vehicle, VehicleCategory, Employee, Branch, Customer, LoyaltyTier;
6.
7. -- =====
8. -- LoyaltyTier Table
9. -- =====
10. CREATE TABLE LoyaltyTier (
11.   TierID INT PRIMARY KEY,
12.   TierName VARCHAR(20) NOT NULL,
13.   DiscountRate DECIMAL(5,2) NOT NULL
14. );
15.
16. -- =====
17. -- Customer Table
18. -- =====
19. CREATE TABLE Customer (
20.   CustomerID INT PRIMARY KEY,
21.   FullName VARCHAR(100) NOT NULL,
22.   Email VARCHAR(100) NOT NULL UNIQUE,
23.   Phone VARCHAR(20),
24.   DriverLicenseNo VARCHAR(30) NOT NULL UNIQUE,
25.   Country VARCHAR(50),
26.   TierID INT NULL,
27.   FOREIGN KEY (TierID) REFERENCES LoyaltyTier(TierID)
28. );
29.
30. -- =====
31. -- Branch Table
32. -- =====
33. CREATE TABLE Branch (
34.   BranchID INT PRIMARY KEY,
35.   Name VARCHAR(100) NOT NULL,
36.   Location VARCHAR(200) NOT NULL,
37.   OperationalStatus VARCHAR(20) CHECK (OperationalStatus IN ('Active',
   'Inactive', 'Closed'))
38. );
```



```

39.
40. -- =====
41. -- Employee Table
42. -- =====
43. CREATE TABLE Employee (
44.     EmployeeID INT PRIMARY KEY,
45.     Name VARCHAR(100) NOT NULL,
46.     Email VARCHAR(100) UNIQUE,
47.     Phone VARCHAR(15),
48.     Position VARCHAR(20) CHECK (Position IN ('Manager', 'Associate')),
49.     BranchID INT NOT NULL,
50.     FOREIGN KEY (BranchID) REFERENCES Branch(BranchID)
51. );
52.
53. -- =====
54. -- VehicleCategory Table
55. -- =====
56. CREATE TABLE VehicleCategory (
57.     CategoryID INT PRIMARY KEY,
58.     Name VARCHAR(20) NOT NULL,
59.     Description TEXT,
60.     DailyRateMultiplier FLOAT NOT NULL DEFAULT 1.0
61. );
62.
63. -- =====
64. -- Vehicle Table
65. -- =====
66. CREATE TABLE Vehicle (
67.     VehicleID INT PRIMARY KEY,
68.     Make VARCHAR(50) NOT NULL,
69.     Model VARCHAR(50) NOT NULL,
70.     Year INT NOT NULL,
71.     Color VARCHAR(30),
72.     LicensePlateNumber VARCHAR(20) NOT NULL,
73.     Status VARCHAR(20) CHECK (Status IN ('Active', 'Inactive', 'Decommissioned')),
74.     FuelType VARCHAR(10) CHECK (FuelType IN ('Diesel', 'Petrol', 'Electric')),
75.     TransmissionType VARCHAR(10) CHECK (TransmissionType IN ('Manual',
    'Automatic')),
76.     BaseRate DECIMAL(10,2) NOT NULL,
77.     BranchID INT NOT NULL,
78.     CategoryID INT NOT NULL,
79.     FOREIGN KEY (BranchID) REFERENCES Branch(BranchID),
80.     FOREIGN KEY (CategoryID) REFERENCES VehicleCategory(CategoryID)
81. );
82.
83. -- =====
84. -- Service Table
85. -- =====
86. CREATE TABLE Service (
87.     ServiceID INT PRIMARY KEY,
88.     Name VARCHAR(100) NOT NULL,
89.     Description TEXT,
90.     Status VARCHAR(20) CHECK (Status IN ('Available', 'Unavailable')),
91.     Type VARCHAR(20) CHECK (Type IN ('Regular', 'Package'))
92. );
93.
94. -- =====
95. -- RegularService Table
96. -- =====
97. CREATE TABLE RegularService (
98.     ServiceID INT PRIMARY KEY,
99.     BasePrice DECIMAL(10,2),

```

```

100.     Currency VARCHAR(10),
101.     Restrictions TEXT,
102.     FOREIGN KEY (ServiceID) REFERENCES Service(ServiceID)
103. );
104.
105. -- =====
106. -- PackageService Table
107. -- =====
108. CREATE TABLE PackageService (
109.     ServiceID INT PRIMARY KEY,
110.     BasePrice DECIMAL(10,2),
111.     Currency VARCHAR(10),
112.     ValidStart DATE,
113.     ValidEnd DATE,
114.     Inclusions TEXT,
115.     Exclusions TEXT,
116.     GracePeriod INT,
117.     FOREIGN KEY (ServiceID) REFERENCES Service(ServiceID)
118. );
119.
120. -- =====
121. -- PricedService Table
122. -- =====
123. CREATE TABLE PricedService (
124.     PricedServiceID INT PRIMARY KEY,
125.     ServiceID INT,
126.     BranchID INT,
127.     Price DECIMAL(10,2),
128.     StartDate DATE,
129.     EndDate DATE,
130.     ApproverID INT,
131.     FOREIGN KEY (ServiceID) REFERENCES Service(ServiceID),
132.     FOREIGN KEY (BranchID) REFERENCES Branch(BranchID),
133.     FOREIGN KEY (ApproverID) REFERENCES Employee(EmployeeID)
134. );
135.
136. -- =====
137. -- Booking Table
138. -- =====
139. CREATE TABLE Booking (
140.     BookingID INT IDENTITY(1,1) PRIMARY KEY,
141.     BookingDate DATE,
142.     CustomerID INT,
143.     BookingStatus VARCHAR(20),
144.     PaymentMethod VARCHAR(50),
145.     Deposit DECIMAL(10,2),
146.     HandledBy INT,
147.     FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
148.     FOREIGN KEY (HandledBy) REFERENCES Employee(EmployeeID)
149. );
150.
151. -- =====
152. -- BookingService Table
153. -- =====
154. CREATE TABLE BookingService (
155.     BookingID INT,
156.     PricedServiceID INT,
157.     ServiceStart DATE,
158.     ServiceEnd DATE,
159.     PRIMARY KEY (BookingID, PricedServiceID),
160.     FOREIGN KEY (BookingID) REFERENCES Booking(BookingID),
161.     FOREIGN KEY (PricedServiceID) REFERENCES PricedService(PricedServiceID)

```

```

162. );
163.
164. -- =====
165. -- BookingVehicle Table
166. -- =====
167. CREATE TABLE BookingVehicle (
168.     BookingID INT,
169.     VehicleID INT,
170.     PRIMARY KEY (BookingID, VehicleID),
171.     FOREIGN KEY (BookingID) REFERENCES Booking(BookingID),
172.     FOREIGN KEY (VehicleID) REFERENCES Vehicle(VehicleID)
173. );
174.
175. -- =====
176. -- Driver Table
177. -- =====
178. CREATE TABLE Driver (
179.     DriverID INT IDENTITY(1,1) PRIMARY KEY,
180.     BookingID INT,
181.     Name VARCHAR(100),
182.     LicenseInfo VARCHAR(100),
183.     ContactNumber VARCHAR(20),
184.     FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)
185. );
186.
187. -- =====
188. -- ExtraService Table
189. -- =====
190. CREATE TABLE ExtraService (
191.     ExtraServiceID INT PRIMARY KEY,
192.     Name VARCHAR(100),
193.     AdvertisedRate DECIMAL(10,2)
194. );
195.
196.
197. -- =====
198. -- BookingExtraService Table
199. -- =====
200. CREATE TABLE BookingExtraService (
201.     BookingID INT,
202.     ExtraServiceID INT,
203.     PRIMARY KEY (BookingID, ExtraServiceID),
204.     FOREIGN KEY (BookingID) REFERENCES Booking(BookingID),
205.     FOREIGN KEY (ExtraServiceID) REFERENCES ExtraService(ExtraServiceID)
206. );
207.
208. -- =====
209. -- ReturnDetails Table
210. -- =====
211. CREATE TABLE ReturnDetails (
212.     ReturnID INT IDENTITY(1,1) PRIMARY KEY,
213.     BookingID INT,
214.     PickupTime DATETIME,
215.     DropOffTime DATETIME,
216.     VehicleCondition TEXT,
217.     LatePenalty DECIMAL(10,2),
218.     DamagePenalty DECIMAL(10,2),
219.     FinalAmount DECIMAL(10,2),
220.     ProcessedBy INT,
221.     DiscountApprover INT,
222.     FOREIGN KEY (BookingID) REFERENCES Booking(BookingID),
223.     FOREIGN KEY (ProcessedBy) REFERENCES Employee(EmployeeID),

```

```

224.         FOREIGN KEY (DiscountApprover) REFERENCES Employee(EmployeeID)
225.     );
226.
227.     -- =====
228.     -- Invoice Table
229.     -- =====
230.     CREATE TABLE Invoice (
231.         InvoiceID INT IDENTITY(1,1) PRIMARY KEY,
232.         BookingID INT,
233.         BaseCharge DECIMAL(10,2),
234.         ExtraCharge DECIMAL(10,2),
235.         LatePenalty DECIMAL(10,2),
236.         DamagePenalty DECIMAL(10,2),
237.         Discount DECIMAL(10,2),
238.         FinalAmount DECIMAL(10,2),
239.         PaymentMethod VARCHAR(50),
240.         FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)
241.     );
2. Data Entry Table
3. ---Insert LoyaltyTier
4. INSERT INTO LoyaltyTier (TierID, TierName, DiscountRate)
5. VALUES (1, 'Silver', 5.00),
6.         (2, 'Gold', 10.00),
7.         (3, 'Platinum', 15.00);
8.
9. ---Insert Customer
10. INSERT INTO Customer (CustomerID, FullName, Email, Phone, DriverLicenseNo,
    Country, TierID)
11. VALUES (1001, 'Jane Doe', 'jane@example.com', '0400000001', 'DL1001', 'Australia',
    2),
12.         (1002, 'John Smith', 'john@example.com', '0400000002', 'DL1002',
    'Australia', NULL);
13. ---Insert Branch
14. INSERT INTO Branch (BranchID, Name, Location, OperationalStatus)
15. VALUES (1, 'Sydney Central', '123 George St, Sydney', 'Active'),
16.         (2, 'Melbourne Hub', '456 Collins St, Melbourne', 'Active');
17. ---Insert Employee
18. INSERT INTO Employee (EmployeeID, Name, Email, Phone, Position, BranchID)
19. VALUES (2001, 'Alice Manager', 'alice@company.com', '0411000001', 'Manager', 1),
20.         (2002, 'Bob Associate', 'bob@company.com', '0411000002', 'Associate', 1);
21. ---Insert VehicleCategory
22. INSERT INTO VehicleCategory (CategoryID, Name, Description, DailyRateMultiplier)
23. VALUES (1, 'Sedan', 'Standard 4-door car', 1.0),
24.         (2, 'SUV', 'Spacious SUV', 1.2);
25. ---Insert Vehicle
26. INSERT INTO Vehicle (VehicleID, Make, Model, Year, Color, LicensePlateNumber,
    Status, FuelType, TransmissionType, BaseRate, BranchID, CategoryID)
27. VALUES (3001, 'Toyota', 'Corolla', 2022, 'White', 'ABC123', 'Active', 'Petrol',
    'Automatic', 70.00, 1, 1),
28.         (3002, 'Mazda', 'CX-5', 2023, 'Blue', 'XYZ789', 'Active', 'Petrol',
    'Automatic', 90.00, 1, 2),
29.         (3003, 'Hyundai', 'Elantra', 2021, 'Silver', 'HYN321', 'Active', 'Petrol',
    'Automatic', 65.00, 1, 1),
30.         (3004, 'Kia', 'Sportage', 2022, 'Red', 'KIA654', 'Active', 'Petrol',
    'Manual', 85.00, 2, 2),
31.         (3005, 'Tesla', 'Model 3', 2024, 'Black', 'TES999', 'Active', 'Electric',
    'Automatic', 120.00, 2, 1);
32. ---Insert Service
33. INSERT INTO Service (ServiceID, Name, Description, Status, Type)
34. VALUES (4001, 'Daily Rental', 'Standard daily rental', 'Available', 'Regular'),
35.         (4002, 'Weekend Package', '3-day weekend package', 'Available', 'Package');
36. ---Insert RegularService

```

```

37. INSERT INTO RegularService (ServiceID, BasePrice, Currency, Restrictions)
38. VALUES (4001, 70.00, 'AUD', 'None');
39. ---Insert PackageService
40. INSERT INTO PackageService (ServiceID, BasePrice, Currency, ValidStart, ValidEnd,
    Inclusions, Exclusions, GracePeriod)
41. VALUES (4002, 200.00, 'AUD', '2025-07-01', '2025-07-31', 'Unlimited km,
    Insurance', 'Fuel', 2);
42. ---Insert PricedService
43. INSERT INTO PricedService (PricedServiceID, ServiceID, BranchID, Price, StartDate,
    EndDate, ApproverID)
44. VALUES (5001, 4001, 1, 70.00, '2025-07-01', '2025-07-31', 2001),
45.         (5002, 4002, 1, 200.00, '2025-07-01', '2025-07-31', 2001);
46. ---Insert Booking
47. INSERT INTO Booking (BookingDate, CustomerID, BookingStatus, PaymentMethod,
    Deposit, HandledBy)
48. VALUES ('2025-07-01', 1001, 'Confirmed', 'Credit Card', 100.00, 2002);
49. ---Insert BookingService
50. INSERT INTO BookingService (BookingID, PricedServiceID, ServiceStart, ServiceEnd)
51. VALUES (1, 5002, '2025-07-10', '2025-07-12');
52. ---Insert BookingVehicle
53. INSERT INTO BookingVehicle (BookingID, VehicleID)
54. VALUES (1, 3002);
55. ---Insert Driver
56. INSERT INTO Driver (BookingID, Name, LicenseInfo, ContactNumber)
57. VALUES (1, 'Jane Doe', 'DL1001', '0400000001');
58. ---Insert ExtraService
59. INSERT INTO ExtraService (ExtraServiceID, Name, AdvertisedRate)
60. VALUES (6001, 'Child Seat', 10.00),
61.         (6002, 'GPS Navigator', 15.00),
62.         (6003, 'Additional Driver', 20.00),
63.         (6004, 'Insurance Upgrade', 25.00);
64. ---Insert BookingExtraService
65. INSERT INTO BookingExtraService (BookingID, ExtraServiceID)
66. VALUES (1, 6002),
67.         (1, 6004);
68. ---Insert ReturnDetails
69. INSERT INTO ReturnDetails (BookingID, PickupTime, DropOffTime, VehicleCondition,
    LatePenalty, DamagePenalty, FinalAmount, ProcessedBy, DiscountApprover)
70. VALUES ( 1, '2025-07-10 09:00', '2025-07-12 18:00', 'Good condition', 0.00, 0.00,
    240.00, 2002, NULL);
71. ---Insert Invoice
72. INSERT INTO Invoice ( BookingID, BaseCharge, ExtraCharge, LatePenalty,
    DamagePenalty, Discount, FinalAmount, PaymentMethod)
73. VALUES ( 1, 200.00, 40.00, 0.00, 0.00, 0.00, 240.00, 'Credit Card');

```

Task 4:

❖ Booking Procedure (QA)

1. Define parameters

Parameter Name	Data Type	Description
@CustomerID	INT	Refers to the customer making the booking. Validated against the Customer table.
@HandledBy	INT	Refers to the employee who is handling the booking. Validated

		against the Employee table.
@PaymentMethod	VARCHAR(50)	Describes how the payment is made (e.g., 'Credit Card', 'Cash'). Stored in the Booking table.
@Deposit	DECIMAL(10,2)	The deposit amount paid by the customer. Stored in the Booking table.
@BookingMethod	VARCHAR(50)	Describes how the booking was made (e.g., 'Online', 'Phone', etc.). (Note: This is declared but not used in your logic.)

Table-Valued Parameters (TVPs)

Parameter Name	Type	Description
@PricedServices	TVP_PricedService	A table of priced service IDs and their corresponding start and end dates for the booking.
@Vehicles	TVP_Vehicle	A table of vehicle IDs selected for the booking. Used to validate availability and insert into booking.
@Drivers	TVP_Driver	A table containing driver details (name, license info, contact). Inserted into the Driver table.
@ExtraServices	TVP_ExtraService	A table of extra service IDs (e.g., insurance, GPS). Optional. Inserted into BookingExtraService.

2. SQL code for stored procedure for Booking Vehicle

```

3. ----BOOKING PROCEDURE
4. DROP TYPE IF EXISTS TVP_PricedService;
5. DROP TYPE IF EXISTS TVP_Vehicle;
6. DROP TYPE IF EXISTS TVP_Driver;
7. DROP TYPE IF EXISTS TVP_ExtraService;
8.
9. -- Table type for Priced Services
10. CREATE TYPE TVP_PricedService AS TABLE (
11.     PricedServiceID INT,

```

```

12.     ServiceStart DATE,
13.     ServiceEnd DATE
14. );
15.
16. -- Table type for Vehicles
17. CREATE TYPE TVP_Vehicle AS TABLE (
18.     VehicleID INT
19. );
20.
21. -- Table type for Drivers
22. CREATE TYPE TVP_Driver AS TABLE (
23.     Name VARCHAR(100),
24.     LicenseInfo VARCHAR(100),
25.     ContactNumber VARCHAR(20)
26. );
27.
28. -- Table type for Extra Services
29. CREATE TYPE TVP_ExtraService AS TABLE (
30.     ExtraServiceID INT
31. );
32. GO
33. Drop procedure usp_CreatingBooking
34. GO
35. CREATE PROCEDURE usp_CreatingBooking
36.     @CustomerID INT,
37.     @HandledBy INT,
38.     @PaymentMethod VARCHAR(50),
39.     @Deposit DECIMAL(10,2),
40.     @BookingMethod VARCHAR(50),
41.     @PricedServices TVP_PricedService READONLY,
42.     @Vehicles TVP_Vehicle READONLY,
43.     @Drivers TVP_Driver READONLY,
44.     @ExtraServices TVP_ExtraService READONLY
45. AS
46. BEGIN
47.     SET NOCOUNT ON;
48.
49.     BEGIN TRY
50.         -----
51.         --Pre-Validations
52.         -----
53.
54.         -- Validate required tables
55.         IF NOT EXISTS (SELECT 1 FROM @PricedServices)
56.         BEGIN
57.             RAISERROR('At least one priced service must be provided.', 16, 1);
58.             RETURN;
59.         END
60.
61.         IF NOT EXISTS (SELECT 1 FROM @Vehicles)
62.         BEGIN
63.             RAISERROR('At least one vehicle must be selected.', 16, 1);
64.             RETURN;
65.         END
66.
67.         IF NOT EXISTS (SELECT 1 FROM @Drivers)
68.         BEGIN
69.             RAISERROR(' At least one driver must be provided.', 16, 1);
70.             RETURN;
71.         END
72.
73.         -- Validate Customer exists

```

```

74.         IF NOT EXISTS (SELECT 1 FROM Customer WHERE CustomerID = @CustomerID)
75.         BEGIN
76.             RAISERROR('Customer not found.', 16, 1);
77.             RETURN;
78.         END
79.
80.         -- Validate Employee exists
81.         IF NOT EXISTS (SELECT 1 FROM Employee WHERE EmployeeID = @HandledBy)
82.         BEGIN
83.             RAISERROR(' Employee (HandledBy) not found.', 16, 1);
84.             RETURN;
85.         END
86.
87.         -- Validate all PricedServices exist
88.         IF EXISTS (
89.             SELECT ps.PricedServiceID
90.             FROM @PricedServices ps
91.             LEFT JOIN PricedService p ON ps.PricedServiceID = p.PricedServiceID
92.             WHERE p.PricedServiceID IS NULL
93.         )
94.         BEGIN
95.             RAISERROR('One or more priced service IDs are invalid.', 16, 1);
96.             RETURN;
97.         END
98.
99.         -- Validate all Vehicles exist
100.        IF EXISTS (
101.            SELECT v.VehicleID
102.            FROM @Vehicles v
103.            LEFT JOIN Vehicle master ON v.VehicleID = master.VehicleID
104.            WHERE master.VehicleID IS NULL
105.        )
106.        BEGIN
107.            RAISERROR('One or more vehicle IDs are invalid.', 16, 1);
108.            RETURN;
109.        END
110.
111.        -----
112.        -- Step 1: Insert Booking
113.        -----
114.
115.        INSERT INTO Booking (BookingDate, CustomerID, BookingStatus,
116.            PaymentMethod, Deposit, HandledBy)
117.        VALUES (GETDATE(), @CustomerID, 'Confirmed', @PaymentMethod,
118.            @Deposit, @HandledBy);
119.
120.        DECLARE @NewBookingID INT = SCOPE_IDENTITY();
121.
122.        -----
123.        -- Step 2: Validate and Insert Priced Services
124.        -----
125.
126.        IF EXISTS (
127.            SELECT 1
128.            FROM @PricedServices ps
129.            JOIN PricedService p ON ps.PricedServiceID =
130.            p.PricedServiceID
131.            WHERE ps.ServiceStart < p.StartDate OR ps.ServiceEnd >
132.            p.EndDate
133.        )
134.        BEGIN

```



```

131.             RAISERROR('One or more service dates are outside the valid
pricing period.', 16, 1);
132.             RETURN;
133.         END
134.
135.         INSERT INTO BookingService (BookingID, PricedServiceID,
ServiceStart, ServiceEnd)
136.         SELECT @NewBookingID, PricedServiceID, ServiceStart, ServiceEnd
137.         FROM @PricedServices;
138.
139.         -----
140.         -- Step 3: Validate Vehicle Availability
141.         -----
142.
143.         IF EXISTS (
144.             SELECT 1
145.             FROM @Vehicles v
146.             JOIN BookingVehicle bv ON v.VehicleID = bv.VehicleID
147.             JOIN Booking b ON b.BookingID = bv.BookingID
148.             JOIN BookingService bs ON bs.BookingID = b.BookingID
149.             WHERE b.BookingStatus = 'Confirmed'
150.             AND EXISTS (
151.                 SELECT 1 FROM @PricedServices ps
152.                 WHERE ps.ServiceStart <= bs.ServiceEnd AND
ps.ServiceEnd >= bs.ServiceStart
153.             )
154.         )
155.         BEGIN
156.             RAISERROR('One or more vehicles are already booked for the
selected period.', 16, 1);
157.             RETURN;
158.         END
159.
160.         -----
161.         -- Step 4: Insert Vehicles, Drivers, Extras
162.         -----
163.
164.         INSERT INTO BookingVehicle (BookingID, VehicleID)
165.         SELECT @NewBookingID, VehicleID FROM @Vehicles;
166.
167.         INSERT INTO Driver (BookingID, Name, LicenseInfo, ContactNumber)
168.         SELECT @NewBookingID, Name, LicenseInfo, ContactNumber FROM
@Drivers;
169.
170.         IF EXISTS (SELECT 1 FROM @ExtraServices)
171.         BEGIN
172.             INSERT INTO BookingExtraService (BookingID, ExtraServiceID)
173.             SELECT @NewBookingID, ExtraServiceID FROM @ExtraServices;
174.         END
175.
176.         PRINT 'Booking created successfully. Booking ID: ' +
CAST(@NewBookingID AS VARCHAR);
177.     END TRY
178.     BEGIN CATCH
179.         DECLARE @ErrMsg NVARCHAR(4000), @ErrSeverity INT;
180.         SELECT @ErrMsg = ERROR_MESSAGE(), @ErrSeverity =
ERROR_SEVERITY();
181.         RAISERROR(@ErrMsg, @ErrSeverity, 1);
182.     END CATCH
183.     END;

```

❖ Return Procedure

1. Define parameters

Parameter	Data Type	Purpose
@BookingID	INT	Identifies the specific booking being returned. Must exist and be in 'Confirmed' status.
@PickupTime	DATETIME	The actual time when the vehicle was picked up. Used for return record tracking.
@DropOffTime	DATETIME	The actual return time when the vehicle was dropped off. Logged in the ReturnDetails table.
@VehicleCondition	VARCHAR(100)	Notes on the condition of the returned vehicle (e.g., "Good", "Scratched bumper", etc.).
@LatePenalty	DECIMAL(10,2)	Any additional charge applied due to late return. Added to final amount.
@DamagePenalty	DECIMAL(10,2)	Charge applied for any damages to the vehicle. Added to final amount.
@ProcessedBy	INT	Employee ID of the staff processing the return. Must exist in the Employee table.
@DiscountApprover	INT (optional)	Employee ID who approved the discount (if any). Can be NULL. Stored in the ReturnDetails.

2, SQL code for stored procedure for Return Vehicle

```

184. CREATE PROCEDURE usp_ProcessReturn
185.     @BookingID INT,
186.     @PickupTime DATETIME,
187.     @DropOffTime DATETIME,
188.     @VehicleCondition VARCHAR(100),
189.     @LatePenalty DECIMAL(10,2),
190.     @DamagePenalty DECIMAL(10,2),
191.     @ProcessedBy INT,
192.     @DiscountApprover INT = NULL
193. AS
194. BEGIN

```

```

195.         SET NOCOUNT ON;
196.
197.     BEGIN TRY
198.         -- Validate booking
199.         IF NOT EXISTS (
200.             SELECT 1 FROM Booking
201.             WHERE BookingID = @BookingID AND BookingStatus = 'Confirmed'
202.         )
203.         BEGIN
204.             RAISERROR('Booking not found or not in a returnable state.',
16, 1);
205.             RETURN;
206.         END
207.
208.         -- Validate employee
209.         IF NOT EXISTS (
210.             SELECT 1 FROM Employee WHERE EmployeeID = @ProcessedBy
211.         )
212.         BEGIN
213.             RAISERROR(' ProcessedBy employee not found.', 16, 1);
214.             RETURN;
215.         END
216.
217.         -- Calculate base and extra charges
218.         DECLARE @BaseCharge DECIMAL(10,2) = (
219.             SELECT SUM(p.Price)
220.             FROM BookingService bs
221.             JOIN PricedService p ON bs.PricedServiceID =
p.PricedServiceID
222.             WHERE bs.BookingID = @BookingID
223.         );
224.
225.         DECLARE @ExtraCharge DECIMAL(10,2) = (
226.             SELECT ISNULL(SUM(e.AdvertisedRate), 0)
227.             FROM BookingExtraService bes
228.             JOIN ExtraService e ON bes.ExtraServiceID = e.ExtraServiceID
229.             WHERE bes.BookingID = @BookingID
230.         );
231.
232.         -- Loyalty discount
233.         DECLARE @DiscountRate DECIMAL(5,2) = (
234.             SELECT ISNULL(t.DiscountRate, 0)
235.             FROM Booking b
236.             JOIN Customer c ON b.CustomerID = c.CustomerID
237.             LEFT JOIN LoyaltyTier t ON c.TierID = t.TierID
238.             WHERE b.BookingID = @BookingID
239.         );
240.
241.         DECLARE @DiscountAmount DECIMAL(10,2) = ROUND(
242.             (@BaseCharge + @ExtraCharge) * @DiscountRate / 100, 2
243.         );
244.
245.         -- Calculate final amount
246.         DECLARE @FinalAmount DECIMAL(10,2) =
247.             (@BaseCharge + @ExtraCharge + @LatePenalty + @DamagePenalty)
- @DiscountAmount;
248.
249.         -- Insert return record
250.         INSERT INTO ReturnDetails (
251.             BookingID, PickupTime, DropOffTime, VehicleCondition,
252.             LatePenalty, DamagePenalty, FinalAmount,
253.             ProcessedBy, DiscountApprover

```

```

254.         )
255.         VALUES (
256.             @BookingID, @PickupTime, @DropOffTime, @VehicleCondition,
257.             @LatePenalty, @DamagePenalty, @FinalAmount,
258.             @ProcessedBy, @DiscountApprover
259.         );
260.
261.         -- Insert invoice
262.         INSERT INTO Invoice (
263.             BookingID, BaseCharge, ExtraCharge, LatePenalty,
264.             DamagePenalty, Discount, FinalAmount, PaymentMethod
265.         )
266.         SELECT
267.             b.BookingID, @BaseCharge, @ExtraCharge, @LatePenalty,
268.             @DamagePenalty, @DiscountAmount, @FinalAmount,
269.             b.PaymentMethod
270.         FROM Booking b
271.         WHERE b.BookingID = @BookingID;
272.
273.         -- Update booking status
274.         UPDATE Booking
275.         SET BookingStatus = 'Serviced'
276.         WHERE BookingID = @BookingID;
277.
278.         -- Reactivate vehicle(s)
279.         UPDATE v
280.         SET v.Status = 'Active'
281.         FROM Vehicle v
282.         JOIN BookingVehicle bv ON v.VehicleID = bv.VehicleID
283.         WHERE bv.BookingID = @BookingID;
284.
285.         -- Message output
286.         PRINT 'Return processed successfully. Final Amount: $' +
287.             CAST(@FinalAmount AS VARCHAR);
288.
289.         IF @LatePenalty > 0 OR @DamagePenalty > 0
290.             PRINT ' Penalties applied: Late Fee = $' + CAST(@LatePenalty
291.                 AS VARCHAR) +
292.                 ', Damage Fee = $' + CAST(@DamagePenalty AS VARCHAR);
293.
294.         IF @DiscountAmount > 0
295.             PRINT 'Loyalty discount applied: $' + CAST(@DiscountAmount
296.                 AS VARCHAR);
297.
298.         END TRY
299.         BEGIN CATCH
300.             DECLARE @ErrMsg NVARCHAR(4000), @ErrSeverity INT;
301.             SELECT @ErrMsg = ERROR_MESSAGE(), @ErrSeverity =
302.                 ERROR_SEVERITY();
303.             RAISERROR(@ErrMsg, @ErrSeverity, 1);
304.         END CATCH
305.     END;

```

TEST CASES

We ran the sp_BookVehicle stored procedure through a set of valid and invalid test cases to determine that they are functioning correctly, enforcing constraints, and catching certain errors. The findings are listed as follows:

Test Case	Input	Expected Outcome	Purpose
1. Booking Successfully	- CustomerID = 1002 - VehicleID = 3003 - PricedServiceID = 5001 (15–16 July) - Valid driver and extras	Booking is created successfully. Print message with new Booking ID.	To verify successful booking with all valid inputs.
2. Vehicle Already Booked (Overlapping Dates)	VehicleID = 3002 already booked for overlapping date range (e.g., 10–12 July) - PricedServiceID = 5002 from 11–13 July	Error: 'One or more vehicles are already booked for the selected period.'	To check that the procedure prevents booking when vehicle is unavailable due to date overlap.
3. Service Dates Outside Pricing Period	- PricedServiceID = 5002 but service dates '2025-06-25' to '2025-06-28' are before its valid start date	Error: 'One or more service dates are outside the valid pricing period.'	To validate the system rejects services booked outside allowed pricing range.
4. Invalid Customer ID	CustomerID = 9999 (nonexistent) - All other inputs valid	Error: 'Customer not found.'	To verify customer validation and ensure bookings cannot be created with invalid or non-existent customers.

SQL code:

1. ---1.Booking Successfully
- 2.
3. DECLARE @Services1 TVP_PricedService;
4. INSERT INTO @Services1 (PricedServiceID, ServiceStart, ServiceEnd)
5. VALUES (5001, '2025-07-15', '2025-07-16');
6. DECLARE @Vehicles1 TVP_Vehicle;
7. INSERT INTO @Vehicles1 (VehicleID)
8. VALUES (3003);
9. DECLARE @Drivers1 TVP_Driver;
10. INSERT INTO @Drivers1 (Name, LicenseInfo, ContactNumber)
11. VALUES ('John Smith', 'DL1002', '0400000002');
12. DECLARE @Extras1 TVP_ExtraService;
13. INSERT INTO @Extras1 (ExtraServiceID)
14. VALUES (6001);
15. EXEC usp_CreatingBooking
16. @CustomerID = 1002,

```

17. @HandledBy = 2002,
18. @PaymentMethod = 'Credit Card',
19. @Deposit = 100.00,
20. @BookingMethod = 'App',
21. @PricedServices = @Services1,
22. @Vehicles = @Vehicles1,
23. @Drivers = @Drivers1,
24. @ExtraServices = @Extras1;
25.
26. ---Vehicle Already Booked (Overlapping Dates)
27. DECLARE @Services2 TVP_PricedService;
28. INSERT INTO @Services2 VALUES (5002, '2025-07-11', '2025-07-13'); -- Overlaps
    with existing booking
29. DECLARE @Vehicles2 TVP_Vehicle;
30. INSERT INTO @Vehicles2 VALUES (3002); -- Already booked for 10–12 July
31. DECLARE @Drivers2 TVP_Driver;
32. INSERT INTO @Drivers2 VALUES ('Jane Doe', 'DL1001', '0400000001');
33. DECLARE @Extras2 TVP_ExtraService;
34. INSERT INTO @Extras2 VALUES (6002);
35.
36. EXEC usp_CreatingBookKing
37. @CustomerID = 1001,
38. @HandledBy = 2002,
39. @PaymentMethod = 'Credit Card',
40. @Deposit = 100.00,
41. @BookingMethod = 'Website',
42. @PricedServices = @Services2,
43. @Vehicles = @Vehicles2,
44. @Drivers = @Drivers2,
45. @ExtraServices = @Extras2;
46.
47. ---2.Service Dates Outside Pricing Period
48. DECLARE @Services3 TVP_PricedService;
49. INSERT INTO @Services3 VALUES (5002, '2025-06-25', '2025-06-28'); -- Before
    valid start
50. DECLARE @Vehicles3 TVP_Vehicle;
51. INSERT INTO @Vehicles3 VALUES (3001);
52. DECLARE @Drivers3 TVP_Driver;
53. INSERT INTO @Drivers3 VALUES ('John Smith', 'DL1002', '0400000002');
54. DECLARE @Extras3 TVP_ExtraService;
55. INSERT INTO @Extras3 VALUES (6003);
56.
57. EXEC usp_CreatingBookKing

```

```

58. @CustomerID = 1002,
59. @HandledBy = 2002,
60. @PaymentMethod = 'Credit Card',
61. @Deposit = 100.00,
62. @BookingMethod = 'Phone',
63. @PricedServices = @Services3,
64. @Vehicles = @Vehicles3,
65. @Drivers = @Drivers3,
66. @ExtraServices = @Extras3;
67. ----4. Invalid Customer ID
68. DECLARE @Services4 TVP_PricedService;
69. INSERT INTO @Services4 VALUES (5001, '2025-07-15', '2025-07-16');
70. DECLARE @Vehicles4 TVP_Vehicle;
71. INSERT INTO @Vehicles4 VALUES (3001);
72. DECLARE @Drivers4 TVP_Driver;
73. INSERT INTO @Drivers4 VALUES ('Ghost User', 'DL9999', '0400000999');
74. DECLARE @Extras4 TVP_ExtraService;
75. INSERT INTO @Extras4 VALUES (6001);
76.
77. EXEC usp_CreatingBookKing
78. @CustomerID = 9999, -- Does not exist
79. @HandledBy = 2002,
80. @PaymentMethod = 'Credit Card',
81. @Deposit = 100.00,
82. @BookingMethod = 'In Person',
83. @PricedServices = @Services4,
84. @Vehicles = @Vehicles4,
85. @Drivers = @Drivers4,
86. @ExtraServices = @Extras4;
87. GO

```

Test - Return Vehicle Stored Procedure

A set of structured test cases was carried out to ensure the rightfulness, soundness and handling of the exceptions of the sp_Processreturn stored procedure. Such tests use realistic conditions of valid and invalid input parameters to simulate how the system can react to different conditions.

The input, anticipated output, and the purpose of each test case have been summed up in the table below. The process guarantees normal operation as well as edge cases (e.g. invalid bookings, inappropriate discounts approval) proper processing to preserve databases integrity and business logic observance.

Test Case	Input	Expected Outcome	Purpose
1. Successful Return with Loyalty Discount, No Penalties	- BookingID = 1 - Gold tier customer (TierID = 2, 10% discount) - No late/damage fees - Valid employee ProcessedBy = 2002	✓ Return processed successfully ✓ Final amount includes 10% loyalty discount ✓ Vehicle status updated to 'Active' ✓ Status = 'Serviced' ✓ Records added to ReturnDetails and Invoice	To verify full successful return flow with discount and clean record
2. Return with Late and Damage Penalties, No Discount	BookingID = 2 - Customer without loyalty tier - LatePenalty = 50, DamagePenalty = 120 - No discount - Valid employee ProcessedBy = 2002	✓ Return processed successfully ✓ Final amount includes all penalties ✓ No discount shown ✓ Records added to ReturnDetails and Invoice ✓ Booking status updated	To validate penalties are included and no discount is applied if customer has no loyalty tier
3. Invalid Booking ID	BookingID = 999 (does not exist) - All other inputs valid	✗ Error: 'Booking not found or not in a returnable state.' Return blocked	To ensure invalid bookings are rejected and error message is shown
4. ProcessedBy Employee Doesn't Exist	BookingID = 1 (valid) - ProcessedBy = 9999 (invalid employee ID)	✗ Error: 'ProcessedBy employee not found.' Return blocked	To verify the procedure validates the employee ID and prevents

			processing with invalid staff
--	--	--	----------------------------------

SQL code:

```

1. --1. Successful Return with Loyalty Discount, No Penalties
2. -- Reset booking status (for testing only)
3. UPDATE Booking
4. SET BookingStatus = 'Confirmed'
5. WHERE BookingID = 1;
6. EXEC usp_ProcessReturn
7.     @BookingID = 1, -- Booking exists, belongs to Gold customer (TierID = 2, 10%
    discount)
8.     @PickupTime = '2025-07-10 09:00',
9.     @DropOffTime = '2025-07-12 18:00',
10.    @VehicleCondition = 'Excellent',
11.    @LatePenalty = 0.00,
12.    @DamagePenalty = 0.00,
13.    @ProcessedBy = 2002,
14.    @DiscountApprover = NULL; -- Associate applying discount for loyal customer
15. --2.Return with Late and Damage Penalties, No Discount
16.
17. INSERT INTO Booking (BookingDate, CustomerID, BookingStatus,
    PaymentMethod, Deposit, HandledBy)
18. VALUES ('2025-07-05', 1003, 'Confirmed', 'Credit Card', 100.00, 2002);
19. INSERT INTO BookingService (BookingID, PricedServiceID, ServiceStart,
    ServiceEnd)
20. VALUES (2, 5001, '2025-07-10', '2025-07-12'); -- Regular service
21. INSERT INTO BookingVehicle (BookingID, VehicleID)
22. VALUES (2, 3005); -- Make sure this vehicle is available
23. INSERT INTO Driver (BookingID, Name, LicenseInfo, ContactNumber)
24. VALUES (2, 'Alex Carter', 'DL3001', '0400000005');
25. UPDATE Booking
26. SET BookingStatus = 'Confirmed'
27. WHERE BookingID = 2;
28. EXEC usp_ProcessReturn
29.     @BookingID = 2,
30.     @PickupTime = '2025-07-10 09:00',
31.     @DropOffTime = '2025-07-13 20:30',
32.     @VehicleCondition = 'Scratched rear bumper',
33.     @LatePenalty = 50.00,
34.     @DamagePenalty = 120.00,
35.     @ProcessedBy = 2002,
36.     @DiscountApprover = NULL;
37.
38. ---2. Invalid Booking ID
39.
40. EXEC usp_ProcessReturn
41.     @BookingID = 999, -- Doesn't exist
42.     @PickupTime = '2025-07-01 10:00',
43.     @DropOffTime = '2025-07-05 16:00',

```

```
44. @VehicleCondition = 'Good',
45. @LatePenalty = 0.00,
46. @DamagePenalty = 0.00,
47. @ProcessedBy = 2002,
48. @DiscountApprover = NULL;
49.
50. ---3.ProcessedBy Employee Doesn't Exist
51. -- Reset booking status (for testing only)
52. UPDATE Booking
53. SET BookingStatus = 'Confirmed'
54. WHERE BookingID = 1;
55. EXEC usp_ProcessReturn
56. @BookingID = 1,
57. @PickupTime = '2025-07-10 09:00',
58. @DropOffTime = '2025-07-12 18:00',
59. @VehicleCondition = 'Normal wear',
60. @LatePenalty = 0.00,
61. @DamagePenalty = 0.00,
62. @ProcessedBy = 9999, -- Invalid employee
63. @DiscountApprover = NULL;
64.
```